

Platforma demo — 15 containere, o singură poartă

Exerciții · `compose-demo.yaml` · nginx pe o instanță EC2

Primești `compose-demo.yaml` : 15 containere, niciunul cu port public. Îl modifici **doar** acolo unde o cerință o spune — nu ca să scapi de o problemă.

Nu primești configurația de nginx. Aia o scrii tu, de la zero.

Fiecare pas are un criteriu de gata. „Pornește” nu e criteriu.

Platforma consumă aproximativ **350 MB de RAM** în repaus. O instanță cu 1 GB ajunge dacă e curată; cu 2 GB lucrezi liniștit.

Contractul — ce trebuie să răspundă la final

Total pe portul **80**, fără număr de port în adresă:

Serviciu	Port loopback	Adresa publică
whoami	8101	/whoami/
hello	8102	/hello/
httpbin	8103	/httpbin/get
adminer	8104	/adminer/
mailpit	8105	/mail/
dozzle	8106	/logs/
filebrowser	8107	/files/
swagger-ui	8108	/docs/
prometheus	8109	/prom/graph
node-exporter	8110	/node/metrics
grafana	8111	/grafana/login
cyberchef	8112	/chef/
uptime-kuma	8113	provocarea 1
it-tools	8114	provocarea 1
postgres	—	fără port, fără rută

Pasul 1 — Instanța

O instanță EC2 Ubuntu curată, cu Docker instalat. Security Group cu exact două reguli inbound: SSH 22 de la *My IP*, HTTP 80 de la oriunde. Nimic altceva, de la început.

CRITERIU DE GATA

```
docker run --rm hello-world
free -h # cel puțin 1 GB disponibil
```

Pasul 2 — Pornește platforma

Urcă `compose-demo.yaml` pe server și pornește-o. Fișierul cere niște variabile care nu există încă; le afli citindu-l. Parolele le generezi, nu le inventezi:

```
openssl rand -base64 24
```

`.env` nu intră niciodată în git. Asigură-te de asta înainte de primul `git add`, nu după.

CRITERIU DE GATA

```
docker compose -f compose-demo.yaml config -q    # nicio iesire
docker compose -f compose-demo.yaml ps          # 15 servicii running
git status                                       # .env nu apare
```

La postgres , coloana PORTS trebuie să fie **goală**.

Pasul 3 — Verifică serviciile înainte de nginx

Pe server, nu de pe laptop. Dacă un serviciu nu răspunde aici, nginx n-are ce să salveze.

```
for p in 8101 8102 8103 8104 8105 8106 8107 8108 \
        8109 8110 8111 8112 8113 8114; do
    printf '%s -> ' $p
    curl -s -o /dev/null -w '%{http_code}\n' -m 5 http://127.0.0.1:$p/
done
```

CRITERIU DE GATA

Toate 14 întorc 200 sau 3xx . Zero 000 (nu ascultă nimeni) și zero 502 .

Pasul 4 — nginx

Ca pachet pe server, nu în container. Site-ul implicit iese din sites-enabled .

CRITERIU DE GATA

```
sudo nginx -t
curl -s http://127.0.0.1/ | head -3
```

Nu trebuie să scrie „Welcome to nginx!”. Dacă scrie, n-ai șters default-ul.

Pasul 5 — Rutează cele 12 rute

Toate serviciile din tabel, în spatele portului 80, fiecare pe ruta ei. Cele două marcate „provocarea 1” le lași deoparte deocamdată.

CRITERIU — un 200 pe HTML NU e criteriu de gata

Deschide fiecare rută în browser, cu DevTools → Network deschis. Pagina trebuie să se vadă **întreagă**: stiluri aplicate, zero linii roșii, zero 404 pe .css sau .js .

O parte din serviciile din listă răspund 200 pe pagina principală și sunt totuși stricate. Le găsești doar în Network.

CRITERIU DE GATA — de pe laptop

```
for r in /whoami/ /hello/ /httpbin/get /adminer/ /mail/ \
        /logs/ /files/ /docs/ /prom/graph /node/metrics \
        /grafana/login /chef;; do
    printf '%-16s -> ' $r
    curl -s -o /dev/null -w '%{http_code}\n' -m 8 http://IP_PUBLIC$r
done
```

12 din 12 pe 200 , plus verificarea din browser de mai sus.

Pasul 6 — Măsoară

Pe server, cu platforma pornită:

```
free -h
docker stats --no-stream
```

CRITERIU DE GATA

În mesajul de predare scrii două numere și un nume: cât RAM cere platforma în total, cât RAM mai e liber pe instanță, și care container consumă cel mai mult.

Pasul 7 — Închide tot ce nu e portul 80

În Security Group rămân doar 22 (My IP) și 80. Din compose nu mai publici nimic în afară de loopback.

CRITERIU DE GATA — de pe laptop

```
curl -sS -m 5 http://IP_PUBLIC/whoami/      # HTML
curl -sS -m 5 http://IP_PUBLIC:8101        # eroare
curl -sS -m 5 http://IP_PUBLIC:5432        # eroare
```

Prima trebuie să meargă, ultimele două trebuie să nu meargă.

Ce predai

- platform-demo.conf — configurația ta, comisă în lectii/L2-server-nginx/platforma-demo/
 - .env.example — numele variabilelor, fără valori
 - compose-demo.yaml, cu modificările pe care a trebuit să le faci
 - cele trei numere din pasul 6, plus răspunsurile în scris la provocări
- .env nu se comite. Dacă apare în git status, te oprești și rezolvi asta întâi.

Când ceva nu merge

Ce vezi	Unde te uiți întâi
502 Bad Gateway	containerul, nu nginx. Pe server: <code>curl 127.0.0.1:PORT/</code> și <code>docker compose ps</code>
404 pe .css / .js	DevTools → Network. Compară calea pe care o cere pagina cu prefixul din <code>location</code>
Pagina se vede fără stiluri	același loc — e același lucru ca rândul de sus
Redirect în buclă, sau 404 pe rădăcina rutei	slash-ul final: la <code>location</code> și la <code>proxy_pass</code> . Nu sunt independente
Browserul se învâрте	Security Group. Timeout = firewall; refused = nu ascultă nimeni
Ai editat, nu se schimbă nimic	<code>sudo nginx -t && sudo systemctl reload nginx</code>

Provocări

- uptime-kuma (8113) și it-tools (8114) **nu pot** sta pe un subpath, oricât configurezi nginx. Află de ce — răspunsul e în HTML-ul pe care îl trimit ele. Apoi fă-le să funcționeze, fără să deschizi niciun port nou și fără să cumperi domeniu.
- dozzle primește `/var/run/docker.sock`, montat `:ro`. Explică de ce mount-ul ăla, chiar și read-only, e practic acces root pe server. Ce ai schimba dacă platforma ar fi a unui client?

3. `/httpbin/get` întoarce în JSON exact URL-ul pe care l-a văzut aplicația. Folosește-l ca să demonstrezi, cu dovada pe ecran, diferența dintre `proxy_pass http://127.0.0.1:8103;` și `proxy_pass http://127.0.0.1:8103/;`. Un paragraf și două ieșiri de `curl`.

4. Oprește instanța EC2 (Stop) și pornește-o din nou (Start). Restul platformei merge, dar **două** servicii se strică. Care, de ce, și ce ai schimba ca să nu se mai întâmple?